

# ATAQUE MitM ARP

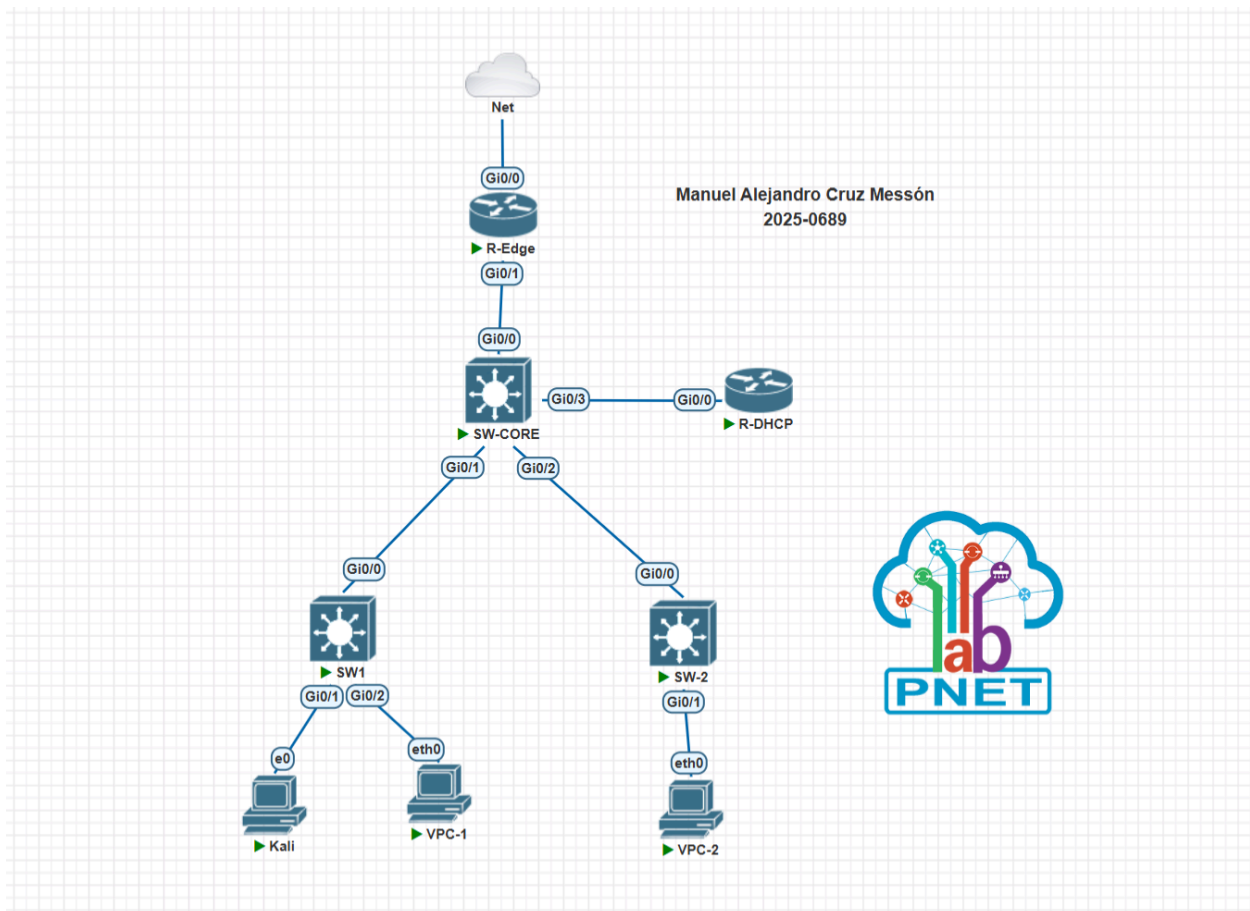
## 1. Objetivo del Laboratorio

El objetivo fundamental de este laboratorio es demostrar de forma práctica y controlada el mecanismo de un ataque de Hombre en el Medio (Man-in-the-Middle o MitM) mediante el envenenamiento de tablas ARP (ARP Spoofing).

El ejercicio busca que los estudiantes u operadores de seguridad entiendan cómo la ausencia de autenticación en el protocolo ARP permite a un atacante interceptar, inspeccionar y eventualmente manipular el tráfico de red local entre una víctima y su puerta de enlace (Gateway), concienciando sobre la importancia de implementar defensas en la capa de enlace de datos.

## 2. Topología de la red

La topología representa una red de laboratorio estructurada bajo una arquitectura jerárquica simple, donde todos los dispositivos internos coexisten en la VLAN 89. La red cuenta con servicios automáticos de asignación de direccionamiento IP (DHCP) administrados por un enrutador dedicado, y salida a redes externas (Internet) a través de un enrutador de borde con traducción de direcciones



## A. Hardware y Dispositivos

La infraestructura física y los nodos que componen la topología se distribuyen según sus roles funcionales en la red:

- **Dispositivos de Enrutamiento (Capa 3):**
  - **R-Edge:** Enrutador de borde perimetral encargado de la salida a redes externas.
  - **R-DHCP:** Enrutador dedicado exclusivamente a la administración y distribución de direccionamiento IP dinámico en la red local.
- **Dispositivos de Conmutación (Capa 2):**
  - **SW-CORE:** Switch central (Núcleo) que interconecta los enrutadores y distribuye el tráfico hacia los switches de acceso.
  - **SW-1 y SW-2:** Switches de acceso encargados de proveer conectividad directa a los nodos finales.
- **Dispositivos Finales (Hosts):**
  - **Kali:** Estación de trabajo orientada del atacante.
  - **VPC-1 y VPC-2:** Computadoras virtuales de escritorio (Virtual PCs) que actúan como usuarios finales de la red.
  - **Net:** Nube que simula el entorno de red externa o Internet.

## B. Componentes de Software

Entorno lógico y sistemas operativos que corren sobre la infraestructura:

- **Sistemas Operativos de Red:** Software basado en emulación de Cisco (IOS) para la gestión y ejecución de protocolos de red (CDP, DHCP, NAT, Routing) en los routers y switches.
- **Sistemas Operativos de Hosts:**
  - **Kali Linux** instalado en la estación atacante.
  - OS ligero (**VPCS**) en las terminales de usuario para pruebas de conectividad básica (Ping, Traceroute).

## C. Segmentación y Parámetros de Red

Definición del direccionamiento lógico, segmentación LAN y salida a Internet:

- **Segmento de Red Interno:** 192.168.89.0/24 (Máscara de subred 255.255.255.0).
- **VLAN Configurada:** VLAN 89, segmento único donde coexisten de forma nativa todos los dispositivos internos, switches (vía SVI) y routers.
- **Puerta de Enlace (Default Gateway):** 192.168.89.254 (Configurada en la interfaz Gi0/1 de R-Edge). Es el nodo encargado de recibir todo el tráfico interno con destino externo y realizar NAT/PAT para darle salida hacia Internet.

#### D. Interfaces Utilizadas

| Dispositivo Origen | Interfaz Local | Dispositivo Destino | Interfaz Remota |
|--------------------|----------------|---------------------|-----------------|
| <b>R-Edge</b>      | Gi0/0          | Net (Nube)          | —               |
|                    | Gi0/1          | SW-CORE             | Gi0/0           |
| <b>R-DHCP</b>      | Gi0/0          | SW-CORE             | Gi0/3           |
| <b>SW-CORE</b>     | Gi0/0          | R-Edge              | Gi0/1           |
|                    | Gi0/3          | R-DHCP              | Gi0/0           |
|                    | Gi0/1          | SW1                 | Gi0/0           |
|                    | Gi0/2          | SW2                 | Gi0/0           |
| <b>SW-1</b>        | Gi0/0          | SW-CORE             | Gi0/1           |
|                    | Gi0/1          | Kali                | e0              |
|                    | Gi0/2          | VPC-1               | eth0            |
| <b>SW-2</b>        | Gi0/0          | SW-CORE             | Gi0/2           |
|                    | Gi0/1          | VPC-2               | eth0            |
| <b>Kali</b>        | e0             | SW1                 | Gi0/1           |
| <b>VPC-1</b>       | eth0           | SW1                 | Gi0/2           |
| <b>VPC-2</b>       | eth0           | SW2                 | Gi0/1           |

### 3. Objetivo del Script

El script mitm-arp.py es una herramienta automatizada escrita en Python utilizando la librería Scapy, diseñada con fines académicos para ejecutar un ataque MitM completo.

Sus objetivos técnicos específicos son:

- **Descubrimiento de Red:** Escanear de forma automatizada la red local mediante peticiones ARP masivas si no se le especifican objetivos manuales.
- **Suplantación de Identidad (Envenenamiento ARP):** Enviar de forma continua respuestas ARP falsas (ARP Replies) tanto a la víctima como al gateway.
  - Le dice a la víctima que la IP del gateway tiene la dirección MAC del atacante.
  - Le dice al gateway que la IP de la víctima tiene la dirección MAC del atacante.
- **Encaminamiento de Tráfico (IP Forwarding):** Modificar los parámetros del kernel de Linux para retransmitir el tráfico interceptado y evitar que la víctima pierda el acceso a internet (lo que delataría el ataque).
- **Inspección de Datos (Sniffing):** Capturar y mostrar en tiempo real los paquetes que transitan por la máquina del atacante, buscando específicamente cadenas de texto claro como peticiones HTTP (GET, POST, credenciales).
- **Restauración del Entorno:** Limpiar y restaurar las tablas ARP originales de los dispositivos afectados una vez que se detiene la ejecución (mediante señales SIGINT / Ctrl+C).

### 4. Parámetros Usados

El script utiliza el módulo argparse para recibir configuraciones desde la línea de comandos. Los parámetros disponibles son:

| Parámetro       | Tipo        | Descripción                                                                                                                 |
|-----------------|-------------|-----------------------------------------------------------------------------------------------------------------------------|
| -i, --interface | Obligatorio | Especifica la interfaz de red local que se utilizará para el ataque (por ejemplo: eth0, wlan0).                             |
| -g, --gateway   | Opcional    | La dirección IP del Gateway/Router. Si se omite, el script intenta leer /proc/net/route o ejecutar ip route para detectarla |

|              |          |                                                                                                                                                                                          |
|--------------|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|              |          | automáticamente.                                                                                                                                                                         |
| -v, --victim | Opcional | La dirección IP del objetivo. Si se omite, el script realiza un escaneo de red utilizando la máscara CIDR detectada y despliega un menú interactivo para seleccionar una víctima activa. |
| --interval   | Opcional | Modifica el tiempo de espera (en segundos) entre cada ciclo de envío de paquetes ARP falsos. Por defecto está configurado en 1.5 segundos.                                               |

## 5. Requisitos para Utilizar la Herramienta

Para que el script funcione correctamente, el entorno debe cumplir con las siguientes condiciones:

- **Privilegios de Root:** El script interactúa directamente con sockets de bajo nivel (Capa 2) y altera configuraciones del kernel (/proc/sys/net/ipv4/ip\_forward). Debe ejecutarse obligatoriamente usando sudo.
- **Sistema Operativo Linux:** Utiliza comandos específicos de Linux (como ip route, ip addr) y la estructura de archivos /proc para la manipulación de red.
- **Python 3 e Intérprete Instalado:** El script está programado bajo la sintaxis de Python 3.
- **Librería Scapy v2.5.0:** Se requiere tener instalada la suite de manipulación de paquetes Scapy.

### Instalación de dependencias

```
pip3 install scapy==2.5.0
```

## 6. Documentación del funcionamiento del script

### Fase 1: Inicialización y Validación de Entorno

- **Comprobación de Privilegios:** El bloque `__main__` verifica que el ID del usuario sea 0 (`os.getuid() != 0`). Si no se ejecuta con `sudo`, el programa termina inmediatamente con un mensaje de error.
- **Procesamiento de Argumentos:** A través de `argparse`, el script captura la interfaz de red (parámetro obligatorio) y las IPs opcionales del gateway y la víctima.

### Fase 2: Auto-detección

Si el usuario no proporcionó las direcciones IP de manera manual, el script ejecuta su lógica de descubrimiento:

- **Detección del Gateway:** La función `get_default_gateway()` intenta abrir y leer el archivo del sistema `/proc/net/route`. Busca la línea que apunta a la interfaz seleccionada y traduce la dirección hexadecimal de la puerta de enlace a formato decimal con puntos (ej. 192.168.1.1). Si falla, ejecuta el comando de Linux `ip route` para extraerla de la salida de texto.
- **Cálculo del Rango de Red:** La función `get_network_range()` interactúa con los sockets del sistema operativo mediante llamadas `ioctl` para obtener la IP local y la máscara de subred de la interfaz. Con estos datos calcula el rango en formato CIDR (ej. 192.168.1.0/24).
- **Escaneo ARP (Descubrimiento de Hosts):** La función `scan_network()` toma el rango CIDR y construye un paquete de difusión (Broadcast) de Capa 2: un encabezado Ethernet dirigido a `ff:ff:ff:ff:ff:ff` combinado con una petición ARP (`ARP(pdst=network_range)`). Utiliza `srp()` (*Send and Receive Packets*) de Scapy para enviar la ráfaga y recolectar las respuestas de todos los equipos encendidos en la red local.
- **Menú de Selección:** `select_victim()` filtra las respuestas eliminando la IP del propio atacante y la del gateway. Muestra una tabla limpia en la terminal y pide al usuario que digite el número de la víctima que desea atacar.

### Fase 3: Resolución de Direcciones MAC

Antes de poder enviar un paquete dirigido a un equipo, el atacante necesita conocer su dirección física (MAC).

- El script llama a `get_mac()` tanto para la IP de la víctima como para la del gateway.

- Envía una petición ARP legítima de tipo Unicast a cada IP. Al recibir la respuesta, extrae y almacena las variables `victim_mac` y `gateway_mac`.
- También obtiene la MAC de la propia máquina atacante mediante `get_if_hwaddr()`.

#### Fase 4: Preparación del Sistema (IP Forwarding)

Para que el ataque no se convierta en una Denegación de Servicio (DoS) donde la víctima se quede sin internet, el script ejecuta la función `enable_ip_forward()`.

- Esto escribe un 1 en la ruta `/proc/sys/net/ipv4/ip_forward`.
- A partir de este momento, el kernel de Linux del atacante actuará como un router, aceptando los paquetes que reciba de la víctima y reenviándolos de forma transparente hacia el gateway real, y viceversa.

#### Fase 5: Interceptación y Análisis en Tiempo Real (Sniffing)

Antes de iniciar el ataque visual, la función `start_sniff()` levanta un hilo secundario en segundo plano (`threading.Thread`) que ejecuta la función `sniff()` de Scapy.

- **Filtro BPF:** Se configura un filtro estricto de captura (`ip host <IP_Víctima> or ip host <IP_Gateway>`) para processar únicamente el tráfico que cruza entre los dos objetivos seleccionados, ignorando el resto de la red.
- **Callback de Paquetes:** Cada paquete capturado es enviado a `packet_callback()`. Esta función desglosa las capas del paquete:
  - Identifica el protocolo de la Capa de Transporte (TCP, UDP o ICMP).
  - Si detecta tráfico TCP en texto plano, busca una capa de datos crudos (Raw).
  - Intenta decodificar los datos en formato UTF-8 y aplica una búsqueda de palabras clave sensibles: "HTTP", "GET", "POST", o "Authorization:". Si encuentra modificaciones, imprime el fragmento de texto directamente en la consola (lo que permite capturar credenciales o URLs visitadas).

#### Fase 6: Bucle Principal de Envenenamiento ARP (Spoofing)

El hilo principal entra en un bucle infinito (`while True`) que ejecuta de forma constante la función `spoof_arp()` cada 1.5 segundos:

- **Paquete hacia la Víctima:** Se envía un paquete ARP de tipo Respuesta (Código de operación `op=2`). En los campos se define que la IP del Gateway (`psrc=gateway_ip`) está asociada a la dirección MAC del atacante (implícito en el origen del paquete).
- **Paquete hacia el Gateway:** Se envía simultáneamente otro paquete ARP indicando que la IP de la Víctima (`psrc=victim_ip`) ahora está en la dirección MAC

del atacante.

- **Frecuencia:** El envío se repite constantemente en el intervalo definido porque los sistemas operativos suelen borrar o actualizar su caché ARP cada pocos minutos, o la propia infraestructura legítima podría enviar paquetes reales que corrijan la tabla temporalmente.

## Fase 7: Finalización Limpia y Restauración (Cleanup)

El script registra manejadores de señales (signal.signal) para interceptar cuando el usuario presiona Ctrl+C (SIGINT) o el sistema envía un SIGTERM. Al activarse, se ejecuta la función cleanup():

- **Deshabilitar Forwarding:** Se escribe un 0 en /proc/sys/net/ipv4/ip\_forward para regresar el sistema a su estado original.
- **Sanar la Red (restore\_arp):** Se envían 5 paquetes ARP legítimos a la víctima y al gateway. Esta vez, el campo hwsrc (*Source MAC*) contiene la dirección física verdadera de cada dispositivo. Esto corrige instantáneamente las tablas caché ARP de ambos objetivos, asegurando que la comunicación vuelva a la normalidad sin dejar la red inestable.
- **Cierre:** El script finaliza su ejecución de forma controlada mediante sys.exit(0).

## 7. Documentación de Contra-medidas

### A. Mitigación en Redes Conmutadas (Capa 2 - Switches)

- **Dynamic ARP Inspection (DAI):** Es la medida más efectiva a nivel corporativo. Los switches validan las peticiones y respuestas ARP contrastándolas con una base de datos confiable (creada mediante DHCP Snooping). Si una respuesta ARP (como las que genera este script) no coincide con la vinculación IP-MAC legítima, el switch descarta el paquete y bloquea el puerto.
- **Port Security:** Limitar el número de direcciones MAC que se pueden aprender en un único puerto del switch para evitar ataques de desbordamiento o suplantaciones masivas.

### B. Mitigación en el Host (Endpoints)

- **Tablas ARP Estáticas:** En entornos críticos con pocos servidores, se puede configurar de forma manual y estática la relación IP-MAC del Gateway mediante el comando: arp -s <IP\_GATEWAY> <MAC\_GATEWAY>. Esto ignora por completo cualquier paquete ARP falso que envíe el script.
- **Software de Detección (Arpwatch / Antidote):** Herramientas locales que monitorean continuamente la actividad ARP y alertan de inmediato al



administrador si se detecta que una dirección IP ha cambiado repentinamente de dirección MAC (la firma clásica del envenenamiento ARP).

### **C. Mitigación en la Capa de Aplicación (Criptografía)**

- **Cifrado de Extremo a Extremo (HTTPS / SSH / VPN):** Aunque el script logre posicionarse con éxito en medio de la comunicación (MitM), si el tráfico va cifrado bajo protocolos seguros (como TLS/HTTPS), el atacante solo verá datos ilegibles. Las líneas del script que buscan palabras clave en texto claro como "Authorization:" o "POST" quedarían completamente inutilizadas.